

Примечания

1. Это лабораторная работа про нагрузочное тестирование информационных систем, не стоит тут применять инструменты микробенчмаркинга
2. Лучше всего подойдёт проект, для которого уже развернули тестовый стенд в рамках ЛР 2, в противном случае придётся разворачивать ещё одно окружение

Лабораторная работа № 4

Задание

Составить набор сценариев тестирования производительности выбранной информационной системы

Требования

1. Производительность (не важно – по памяти на сценарий, по времени обработки запроса на сервере, по задержке из-за сериализации объектов, по количеству полных gc-пауз если это jvm, по времени warm-up периода при старте приложения) оценивается статистической величиной – требуется набрать достаточно большое количество повторений, чтобы получить объективную картину
2. Набор операций должен имитировать поведение гипотетических пользователей: логин -> выполнение каких-то действий -> ожидание. Не просто спам get/post, нужно продумать сценарий и настроить его запуск в N виртуальных пользователей с целью оценить реальную пропускную способность на выбранных ресурсах
3. Требуется проверить как минимум три сценария:
 - a. постепенное увеличение нагрузки с наблюдением за числом отказов / временем отклика / пропускной способностью / использованием ресурсов: цель найти когда система уходит в отказ и когда у неё стабильная работа на выделенных ресурсах
 - b. перегруз / стресс-тест и отслеживание времени восстановления, когда после перегруза пошла нормальная нагрузка
 - c. стабильная нагрузка в течении долгого времени – для анализа потребления ресурсов
4. Реальные пользователи так же могут выполнять внештатные действия – провести тест из пункта 3.с с 10% и 20% ошибочных операций
5. Испытаний будет много, надо добиться равных условий на каждую попытку, то есть следует использовать отдельный докер-образ на каждый прогон, но с одной и той же конфигурацией
6. Измерение времени (задержек) выполнения операций всегда должно включать в себя:
 - a. график измеряемой величины во времени
 - b. распределение величины по перцентилям по числу запросов
 - c. гистограмму распределения
 - d. требуемые перцентили: 0.5, 0.75, 0.9, 0.95. 0.99

7. Кроме времени задержек надо измерить пропускную способность – самый частый вариант это RPS (requests per second), но в зависимости от типа системы это может быть и что-то ещё, например для систем обработки речи это обычно seconds per seconds, для систем хранения данных это bytes per second, для СУБД это может transactions per second или query per second. Все эти величины так же стоит оценивать не только через средние и медианные значения
8. Требуется снимать данные об утилизации ресурсов разными компонентами – CPU, RAM, чтение / запись на диски (если используется GPU тоже учесть в отчете); в отчете также надо указывать информацию в формате “компонент – min/max/medium RAM, min/max/medium CPU, ...” – итог можно сохранить в json (или другой формат с иерархической структурой); также требуется составить графики утилизации этих ресурсов; рекомендуется использовать Prometheus (<https://prometheus.io/>) или аналог для сбора статистики и отрисовки графиков
9. Стандартные утилиты такие как iostat, uptime можно использовать для самопроверки
10. Использовать готовые инструменты генерации нагрузки (k6, pandora, gatling, jmeter, locust) – нормально
11. Реализовывать свой генератор – тоже нормально
12. Т.к. средства мониторинга, генератор и целевое приложение скорее всего будут на одной машине – нужно избежать конкуренции всего этого как минимум за одни и те же ядра CPU